REVIEW 1: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 1: Introduction

1.1 Project Overview

The evolution of Command Line Interfaces (CLIs)

Command Line Interfaces (CLIs) have served as the foundational pillar of computing and system administration since the dawn of modern operating systems, originating from early teleprinter terminals, the UNIX shell (sh), and modern POSIX environments such as Bash and Zsh. CLIs offer unmatched speed, deterministic scripting capabilities, and direct low-level control over kernel resources, processes, and storage subsystems. However, the operational power of traditional CLIs is tied to an explicit syntax model: users must memorize exact command names, nuanced flags, and order-dependent arguments.

The shift toward Natural Language Processing (NLP) in system administration

Recent advancements in Natural Language Processing (NLP) and Large Language Models (LLMs) have sparked a paradigm shift in human-computer interaction. Instead of burdening system operators and developers with memorizing complex argument vectors, modern system tooling is moving toward 'Agentic Shells'—intelligent agents capable of understanding high-level human intent and translating natural language into safe, verifiable system calls. Within this ecosystem, the Smart Archive Agent serves as a specialized, autonomous specialist dedicated to the storage and archiving domain, providing safe file packaging, compression, decompression, integrity auditing, and cryptographic protection on Ubuntu Linux.

1.2 Problem Statement

Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Traditional archiving and compression tools in Linux (e.g., tar, gzip, bzip2, xz, zip, and gpg) exhibit distinct, often inconsistent syntax conventions. For example, creating an xz-compressed tarball requires remembering 'tar -cJf archive.tar.xz folder', whereas gzip requires '-czf' and bzip2 requires '-cjf'. In contrast, file-level compression utilities like xz or gzip compress in-place and delete the original file unless explicit piping flags ('-c') are passed. Furthermore, symmetric encryption with GPG introduces

additional parameters ('--symmetric', '--cipher-algo AES256'). Users who infrequently handle these tasks face significant friction and cognitive load.

Risk of executing incorrect commands due to human error

Executing erroneous archive commands in shell environments carries acute operational risks. A common mistake is argument inversion (e.g., swapping source and destination paths in tar), which can completely overwrite critical source directories with empty archives. Similarly, unvetted extraction commands can trigger path traversal vulnerabilities (Zip Slip) or overwrite existing production files. The Smart Archive Agent addresses these risks by providing a safe translation layer that dispatches only into bounded, pre-validated functions with explicit path verification and strict non-overwrite rules.

1.3 Objectives

- **Lightweight NL-to-CLI Translation:** To build a responsive, lightweight natural-language interpretation engine that parses natural language instructions into structured operations without requiring heavy third-party runtime dependencies.
- **Predefined Operation Boundaries:** To enforce safety by mapping requests strictly to predefined tool functions (create, extract, list, inspect, compress, decompress, encrypt, decrypt) rather than executing arbitrary shell command strings.
- **Model Context Protocol (MCP) Architectural Alignment:** To design tool definitions, input schemas, and execution workflows in alignment with the Model Context Protocol (MCP) standards, allowing seamless future integration with broader agent networks.
- **Security & Guardrail Integration:** To implement robust path validation, prevent Zip Slip directory traversal attacks, prevent accidental file overwrites, and mandate AES-256 GPG encryption for sensitive storage.
- **Structured Feedback & Auditing:** To return actionable success messages, output artifacts, entry listings, and descriptive error diagnostics directly to the user.

1.4 Scope and Limitations

- **Supported Operating Systems:** The Smart Archive Agent is natively targeted for Ubuntu Linux and Windows Subsystem for Linux (WSL 2). It leverages standard Linux binaries (tar, gzip, bzip2, xz, file, gpg, zip, unzip) and Python 3.14.
- **Terminal Environments:** Operates within standard terminal shells (Bash, Zsh) as an interactive REPL or direct CLI command tool.
- **Administrative Privileges:** The agent runs strictly with ordinary user privileges. It explicitly avoids requiring 'sudo' or root escalation, eliminating the possibility of root-level filesystem damage.
- **Domain Boundaries:** The agent is strictly bounded to file packaging, compression, and encryption. Unrelated queries (e.g., process management, network configuration, general Q&A) are identified and safely rejected.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs in structured text and code synthesis

Recent frontier models (such as GPT-4, Claude 3.5, and Gemini Pro) have demonstrated remarkable capabilities in translating unstructured natural language into structured outputs, API payloads, and executable code. Function calling and structured tool outputs enable language models to extract entities from natural text and populate predefined JSON schemas with high fidelity. In system administration, this capability enables flexible intent detection across diverse phrasings and languages.

Challenges with LLM hallucination in system-level execution

Despite their linguistic proficiency, LLMs suffer from probabilistic hallucinations—generating non-existent file paths, fabricating invalid command-line flags, or selecting destructive operations. In operating systems, executing an unvetted LLM-generated shell command directly in 'sh' or 'bash' is fundamentally insecure. To mitigate this hazard, our project adopts a strict 'Separation of Concerns': intent interpretation is kept separate from execution. The agent utilizes an intent router with schema-enforced tool calls, ensuring that every argument is verified by deterministic validation guards before reaching the OS.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP (Clients, Servers, and Tools)

The Model Context Protocol (MCP), open-sourced by Anthropic, is an open standard designed to connect AI applications with external tools, data sources, and execution environments. The MCP architecture comprises three key entities: (1) MCP Clients (the host application or agent orchestrator), (2) MCP Servers (lightweight programs exposing specialized capabilities), and (3) MCP Tools (individual functions declared with JSON Schema parameter specifications). MCP establishes clean contracts for discovery, input validation, and execution.

Why MCP is preferred over traditional rigid API tooling for local system access

Traditional system scripting interfaces rely either on brittle text-based CLI parsing or bespoke, proprietary RPC adapters. MCP provides standardized discovery, dynamic capability negotiation, and uniform error reporting. By adopting MCP-aligned tool interfaces, the Smart Archive Agent can run as a standalone terminal specialist today and seamlessly plug into an enterprise-wide multi-agent shell tomorrow without altering its core archiving and security implementations.

2.3 Existing Solutions

A comparative study of current commercial and open-source terminal assistance solutions reveals distinct architectural philosophies:

Solution	Primary Paradigm	Security / Sandboxing	Relevance to this Project

Traditional Bash	Manual command-line execution	User permissions only; no syntax safety or guardrails.	Reliable system backends, but requires exhaustive syntax memorization.
Warp AI	AI-assisted terminal emulator	Suggests shell commands; user must review before executing in terminal.	Demonstrates natural-language interaction in terminals, but executes arbitrary shell scripts.
GitHub Copilot CLI	NL-to-shell command synthesizer	Explains generated bash command; requires manual user confirmation.	Illustrates LLM-driven command generation; vulnerable to flag hallucinations.
Smart Archive Agent (Ours)	Domain-bounded Agentic Specialist	Strict function dispatch; path validation; Zip-Slip prevention; non-root execution.	Combines natural language ease with deterministic guardrails in a specialized storage domain.

References

[1] Anthropic, 'Model Context Protocol (MCP) Specification,' 2024. [Online]. Available: https://github.com/modelcontextprotocol/docs

[2] OpenAI, 'Function Calling and Structured Outputs Guide,' 2024. [Online]. Available: https://developers.openai.com/api/docs/guides/function-calling

[3] Free Software Foundation, 'GNU Tar Manual: Tape Archiving Utility,' GNU Project, 2023. [Online]. Available: https://www.gnu.org/software/tar/manual/

[4] Warp Technologies, 'Warp: The AI-Powered Terminal User Guide,' 2024. [Online]. Available: https://docs.warp.dev/

[5] GitHub Inc., 'GitHub Copilot in the CLI Documentation,' 2024. [Online]. Available: https://docs.github.com/en/copilot/using-github-copilot-in-the-cli

REVIEW 2: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell

Project Title: Smart Archive Agent

Team Details: Team 12

Team Members: Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 3: System Architecture & Design

3.1 High-Level Architecture

The User-to-Shell Loop: User Input → LLM Processing → Tool Call → System Execution → Feedback

The Smart Archive Agent is Agent #12 of the class-wide Agentic Shell. In the shell, the base shell's LLM reads the user's sentence, chooses one of our tools and its arguments, and our code performs the real operating-system work. The agent can also run on its own, in which case a built-in rule-based planner performs the "LLM processing" stage and produces the same tool calls. Both paths reach the operating system only through one validated entry point, execute_tool().

Loop stage	In the Agentic Shell	When the agent runs on its own
User Input	The user types into the base shell.	The user types at the agent's You: prompt (main.py).
LLM Processing	The shell's LLM reads our tool definitions (get_tools()) and picks tools by function calling.	intent.py and planner.py turn the sentence into an ordered plan of tool calls, offline.
Tool Call	The shell calls execute_tool(name, arguments).	Each plan step is passed to execute_tool().
System Execution	tar, gzip, bzip2, xz and gpg run through subprocess.run() with argument lists.	Same.
Feedback	A result dictionary is returned to the LLM as the tool result.	The result is printed: done lines, output path, entries or error.

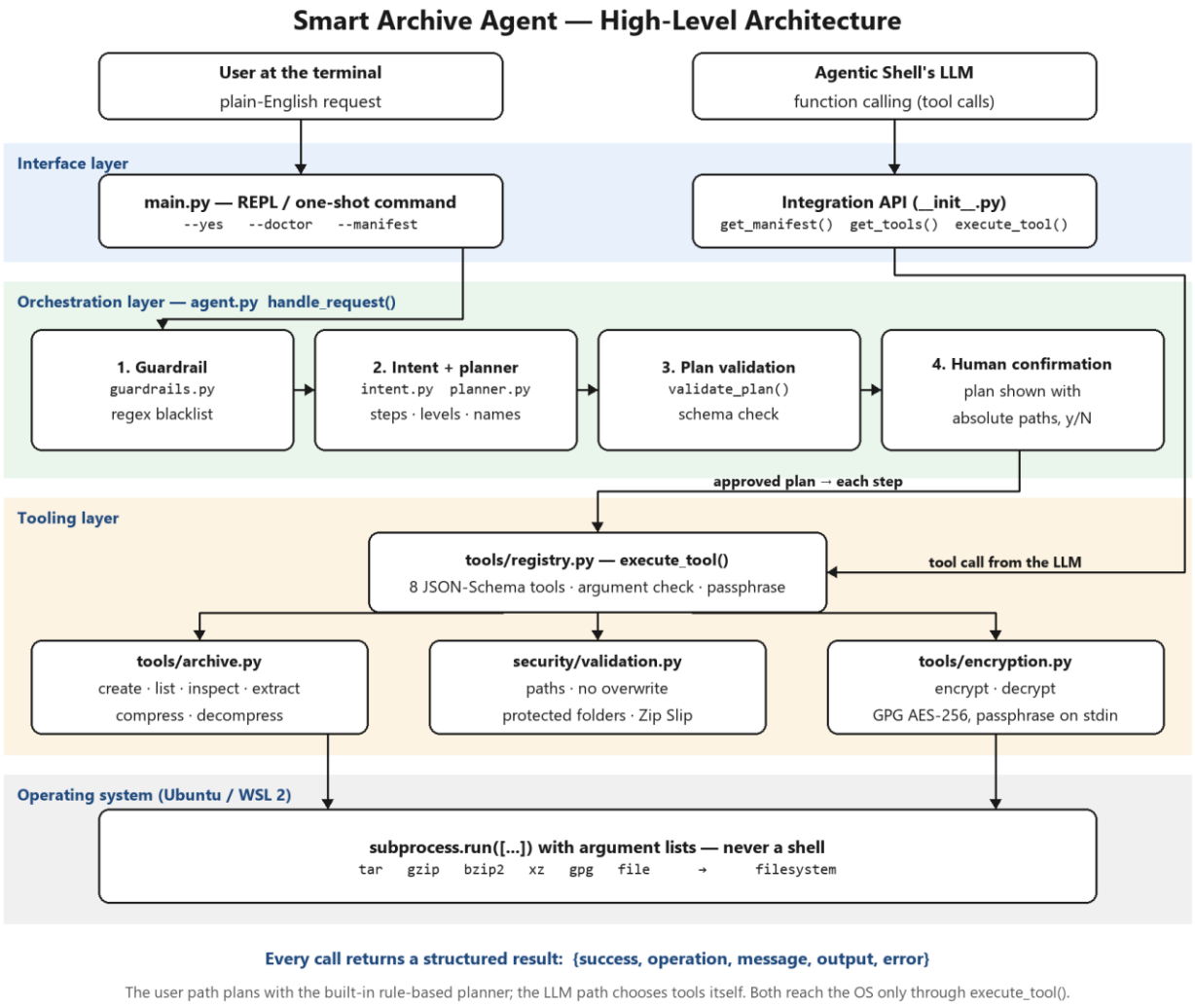


Figure 3.1: High-Level System Architecture of the Smart Archive Agent.

Architecture explanation. Figure 3.1 shows the two entry points at the top: a user at the terminal and the Agentic Shell's LLM. The user path goes through the orchestration layer: a guardrail, the intent recognizer and planner, plan validation and human confirmation. The LLM path skips planning, because the LLM has already chosen the tool, and goes straight to the tool registry. In both cases execute_tool() checks the call against its JSON Schema, the security module re-validates every path, and only then do the archive and encryption modules start Linux utilities. Every call returns the same structured result.

3.2 Component Design

3.2.1 The Interface Layer: Custom REPL (Read-Eval-Print Loop) Environment

The interface layer is implemented in smart_archive_agent/main.py and the run() loop in agent.py. It offers an interactive REPL (a banner, a You: prompt, the proposed plan with absolute paths, a y/N confirmation, a hidden passphrase prompt and formatted results) and a one-shot command mode for scripts. Ctrl+C and end-of-input exit cleanly, and the exit code is 0 on success and 1 on failure.

```
python3 -m smart_archive_agent                          # interactive REPL
python3 -m smart_archive_agent "archive ./src"          # one request, asks before writing
python3 -m smart_archive_agent --yes "back up ./src"    # one request, no confirmation
python3 -m smart_archive_agent --doctor                 # checks tar, gzip, bzip2, xz, gpg,
file
python3 -m smart_archive_agent --manifest               # agent description for the shell's
router
```

For integration, the package itself is part of the interface: __init__.py exports get_manifest(), get_tools(), execute_tool() and handle_request(), and agent.json describes the agent to the shell's router (what it handles, what it does not, and what it needs installed).

3.2.2 The Orchestration Layer: Intent Routing

The template lists LangChain, LlamaIndex or a raw MCP client for this layer. We implemented the orchestration directly in Python (agent.py, handle_request()) without a framework, for three reasons: the agent must install on the teacher's machine with no extra packages, every decision must be predictable and auditable for safety review, and open-ended language understanding is already the base shell's LLM's job. The pipeline runs in a fixed order:

Stage	Module	What it does
1. Guardrail	security/guardrails.py	Matches the raw text against a regex blacklist of destructive commands before anything else.
2. Intent recognition	intent.py	Removes file paths from the text, then matches whole-word verb vocabularies in a fixed priority order (decrypt, encrypt, extract, decompress, list, compress, archive). Out-of-scope requests stop here.
3. Planning	planner.py	Builds ordered steps: folders are archived first; "heavily" → level max, "quickly" → fast; missing output names are chosen next to the source; an unencrypted archive the agent created is removed after encryption.
4. Plan validation	planner.validate_plan()	Checks every step against the tool schemas; a clean-up step may delete only a file created earlier in the same plan.
5. Confirmation and execution	agent.run_plan()	Shows the plan, waits for y, runs the steps in order, and stops at the first failure, reporting completed steps.

3.2.3 The Tooling Layer: Safe System Functions

The tooling layer exposes eight archive functions as tools. Each tool is declared in tools/registry.py with a name, a description written for an LLM, and a JSON Schema for its parameters, which is the same structure an MCP server uses to declare tools. get_tools() returns these declarations in generic, OpenAI-compatible or Anthropic format, and execute_tool() runs one call. The tools are not yet served over the

MCP protocol itself; that wrapper is listed as future work. File-management functions for other domains, system monitoring and network configuration belong to other agents of the shell.

Tool	Python function	Linux backend	Key arguments
create_archive	archive.create_archive()	tar with gzip / bzip2 / xz (tar -I), or Python zipfile	source_path, output_path, compression, level
list_archive	archive.list_archive()	tar -tf, zipfile	archive_path
inspect_archive	archive.inspect_archive()	file -b, tar -tf, zipfile	archive_path
extract_archive	archive.extract_archive()	Python tarfile / zipfile after a full member check	archive_path, destination_path
compress_file	archive.compress_file()	gzip, bzip2, xz	source_path, output_path, algorithm, level
decompress_file	archive.decompress_file()	gzip -d, bzip2 -d, xz -d	source_path, output_path
encrypt_file	encryption.encrypt_file()	gpg --symmetric --cipher-algo AES256	source_path, output_path (+ passphrase)
decrypt_file	encryption.decrypt_file()	gpg --decrypt	source_path, output_path (+ passphrase)

Compression levels: fast = gzip -1 / xz -1, normal = gzip -6 / xz -6, max = gzip -9 / bzip2 -9 / xz -9e.

3.2.4 Result Handling & Diagnostics

Every tool and every request returns one dictionary: success (bool), operation (str), message (str, for people), output (path produced), plus entries for listings, steps and plan for multi-step requests, missing when a detail could not be found, and error on failure. execute_tool() never raises: exceptions inside a tool are turned into success = false with an error message, so neither the REPL nor the base shell can be crashed by a failing operation. The same dictionary can be sent back to an LLM as the tool result with json.dumps().

3.3 Security & Guardrail Framework

3.3.1 The "Human-in-the-Loop" Validation Mechanism

No file is written until a person has seen and approved the complete plan. Before any step that writes, the agent prints every step with absolute source and destination paths and waits for y; anything else cancels with "Nothing was changed". Read-only requests (list, inspect) run without asking. Passphrases for encryption and decryption are always typed by the person at a hidden prompt (asked twice when encrypting); they never come from an LLM. When the agent is embedded in the base shell, handle_request(confirm=…, passphrase=…) lets the shell show the plan and collect the passphrase in its own interface.

3.3.2 Command Sanitization and Regex-Based Blacklisting of Destructive Commands

User text is never executed as a command. Utilities are started with subprocess.run() and an argument list (never shell=True), so characters such as ; | $( ) are passed as plain data. In addition, the raw request is checked against a regex blacklist before planning, so a destructive request is refused outright instead of being partially carried out:

Pattern (examples)	Refused as
rm -rf …, rm -fr …	recursive forced delete
mkfs, mkfs.ext4	filesystem formatting
dd if=…	raw disk write
:(){ :|:& };:	fork bomb
> /dev/sda	raw block-device overwrite
chmod -R 777 /	world-writable root filesystem
shutdown, reboot, poweroff, halt	system power control
format / wipe / erase … disk, drive, everything	disk wipe request
delete everything, delete all files	mass deletion request
; rm, | bash, `…`, $(…)	shell command injection
a bare / as a path	operation on the whole root filesystem

Further guards apply inside every tool, whichever path the call came from:

- **Protected locations:** never writes into /bin, /boot, /dev, /etc, /lib, /lib64, /proc, /sbin, /sys, /usr or /var/lib.
- **No overwrites:** an existing output is never replaced; source and output must differ; an archive cannot be written inside the folder being archived.
- **Zip Slip prevention:** every member of an archive is resolved against the destination and must stay inside it; symbolic and hard links inside tar archives are refused; the destination folder is created only after the whole archive passes.
- **Schema validation of tool calls:** unknown tools, missing or extra arguments, wrong types and invalid values (for example level = "ultra") are rejected before anything runs.
- **Least privilege:** the agent runs as a normal user and never uses sudo.

Chapter 4: Implementation Methodology

4.1 Development Environment & Tech Stack

Item	Choice
Programming Language	Python. The code uses Python 3.8+ syntax and only the standard library (subprocess, tarfile, zipfile, getpass, json, re, pathlib); developed and tested with Python 3.14.4.
Operating System	Ubuntu 26.04.1 LTS on Windows Subsystem for Linux 2 (Windows 11). Targets any Ubuntu or Debian machine.
LLM Provider	Provided by the base Agentic Shell, which owns the LLM. get_tools() exports our tool definitions in OpenAI-compatible format (also accepted by local Ollama) and Anthropic Claude format. The agent itself makes no LLM calls and needs no API key; offline requests use the built-in planner.
SDKs	None required. The MCP Python SDK is not used; the tool declarations follow the same JSON Schema structure, so an MCP server wrapper can be added without changing the tools.
System Utilities	GNU tar 1.35, gzip, bzip2, XZ Utils 5.8.3, GnuPG 2.4.8, file.
Testing	pytest 9.1.1 (58 automated tests) and benchmarks/evaluate.py for accuracy, latency and error recovery.
Packaging & Integration	pyproject.toml (pip install from GitHub, smart-archive-agent command), agent.json manifest, INTEGRATION.md and a runnable integration example.
Tools	Visual Studio Code with the WSL extension, Git and GitHub.

4.2 Step-by-Step Implementation

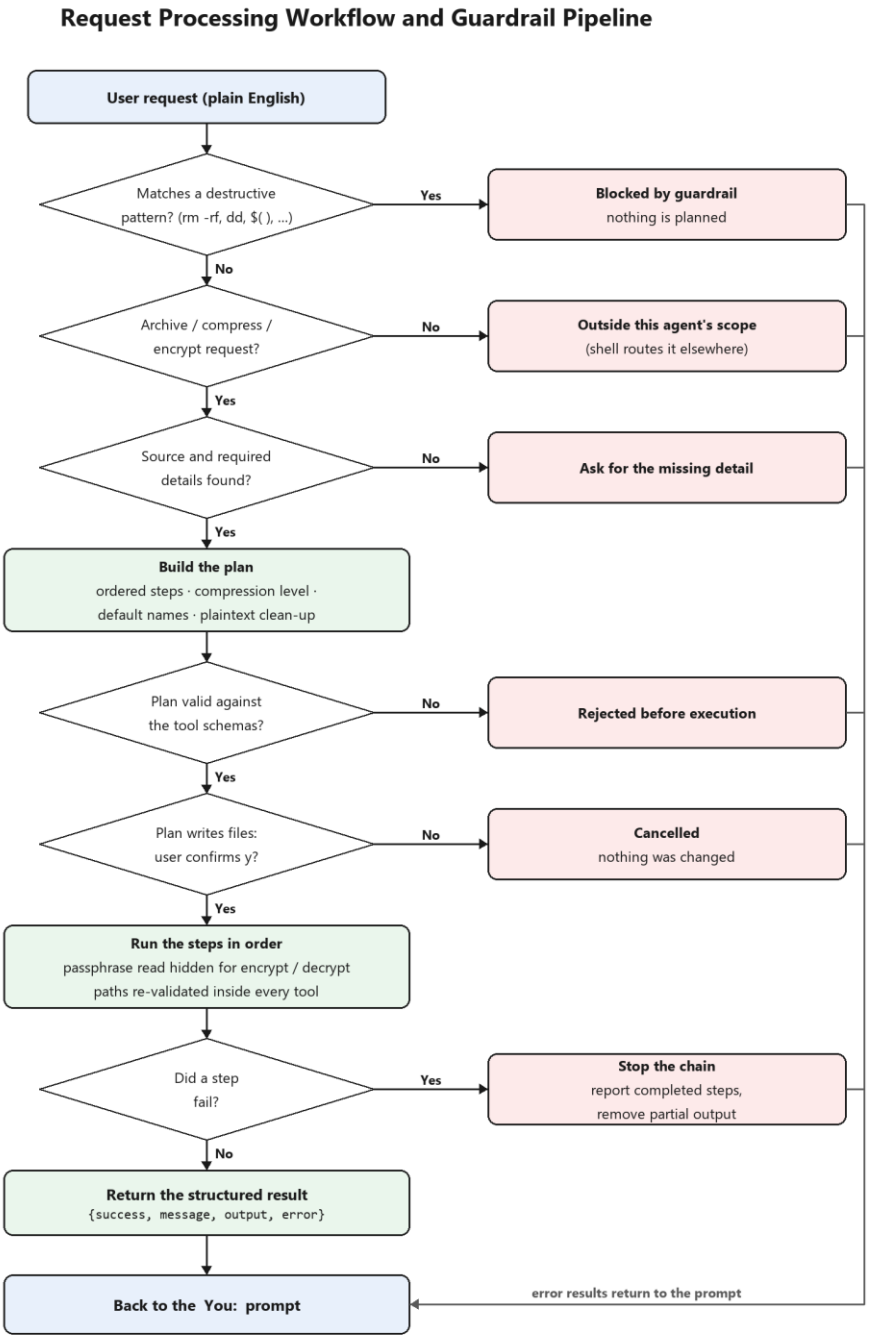


Figure 4.1: End-to-End Request Processing Workflow and Guardrail Pipeline.

Figure 4.1 shows how one request travels through the agent. Each decision has a safe exit on the right (blocked, out of scope, missing detail, rejected, cancelled, or a failed step) that returns a structured error and goes back to the prompt without changing any file. The implementation was built in four steps.

Step 1: Setting up the base CLI prompt loop

main.py parses the command line (interactive, one-shot, --yes, --doctor, --manifest, --version) and agent.run() implements the REPL: read a line, pass it to handle_request() with a terminal confirmation and a hidden passphrase prompt, print the result with print_result(), and repeat until exit, Ctrl+C or end of input. The --doctor check lists any missing Linux tool with the exact apt install command, so the agent can be checked quickly on any lab machine.

Step 2: Building the MCP-style tool layer to expose safe system functions

archive.py and encryption.py implement the eight operations as plain Python functions that validate their paths and return result dictionaries. tools/registry.py then declares each one for function calling (TOOL_SCHEMAS) and provides execute_tool(), the only entry point that runs a tool. The template's examples list_directory and get_system_stats correspond in our domain to list_archive and inspect_archive, which read an archive without changing anything. A prefix option (smart_archive__create_archive) keeps our tool names unique among the 35 agents.

Step 3: Prompt engineering for exact tool matching and JSON format enforcement

For the LLM path, every tool description and parameter description is written so that a model can choose the right tool and values from everyday wording, and JSON Schema enforces the format:

```
"level": {"type": "string", "enum": ["fast", "normal", "max"],
          "description": "Compression effort. Use 'max' when the user says heavily, maximum
                          or as small as possible; 'fast' for quick; otherwise 'normal'."}
"required": ["source_path", "output_path"],   "additionalProperties": false
```

check_arguments() enforces the schema on every call (unknown tool, missing or extra argument, wrong type, value outside the enum), and execute_tool() also accepts the JSON string that LLM APIs return. For the offline path, intent.py matches whole words in a fixed priority order after removing file paths from the text, so "zip" is not found inside "gzip" and "tar" is not found inside "backup.tar.xz". Recognition accuracy was measured on phrasings never used for tuning (Review 3, Section 5.2.1).

Step 4: Implementing the command execution wrapper and output formatter

All utilities run through one wrapper that uses an argument list, captures output, applies a 600-second timeout, closes standard input and turns failures into readable errors:

```
subprocess.run(["tar", "-I", "xz -9e -T0", "-cf", output, "-C", parent, name],
               check=True, capture_output=True, text=True,
               timeout=600, stdin=subprocess.DEVNULL)

# passphrase is sent on standard input, never as an argument
gpg --batch --yes --no-symkey-cache --pinentry-mode loopback \
    --passphrase-fd 0 --symmetric --cipher-algo AES256 \
    --output <file>.gpg <file>
```

run_plan() executes the steps in order, records which files each step produced (so the clean-up step can delete only those), stops at the first failure and removes any partial output. print_result() formats the dictionary for the terminal as done lines, the output path, archive entries and errors. Testing and measured results of this implementation are reported in Review 3 (Chapter 5).

References

[1] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[2] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] JSON Schema, "JSON Schema specification." Available: https://json-schema.org/

[5] OpenAI, "Function calling guide." Available: https://platform.openai.com/docs/guides/function-calling

[6] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Python Software Foundation, PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

REVIEW 3: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell

Project Title: Smart Archive Agent

Team Details: Team 12

Team Members: Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 5: Testing, Evaluation, and Results

5.1 Test Cases & Scenarios

The Smart Archive Agent was tested in three complementary ways: (1) an automated pytest suite of **58 tests** that runs every tool against real files with the real Linux utilities, (2) a labelled set of natural-language requests used to measure intent accuracy (Section 5.2), and (3) a live rehearsal of the complete demonstration in an interactive terminal (Chapter 7). All results in this chapter were produced on Ubuntu 26.04.1 LTS on WSL 2, Python 3.14.4, GNU tar 1.35, XZ Utils 5.8.3, GnuPG 2.4.8, 16 logical CPUs. The scenarios below follow the three categories of the review template.

Test file	Tests	What it verifies
tests/test_intent.py	7	Intent recognition and mapping of paths to tool arguments.
tests/test_end_to_end.py	8	Path validation, tar.xz create/list/inspect/extract round trip, bit-exact xz compression and decompression, overwrite protection, checksums.
tests/test_agent.py	37	Chained plans, compression levels, guardrails, protected folders, plan validation, cancellation, tool schemas, invalid model tool calls, zip round trip, GPG encrypt/decrypt with right and wrong passphrase, Zip Slip.
tests/test_integration.py	6	Integration contract for the Agentic Shell: manifest and agent.json, prefixed tool names, JSON-string arguments, command line, runnable integration example.
Total	58	58 passed (python3 -m pytest -q, Figure 7.6).

5.1.1 Simple Intents

A simple intent maps to one tool call. The template's example ("Find all PDFs created yesterday") belongs to the Semantic File Search agent; the equivalent single-step requests for this agent are archive, compress, inspect, extract, encrypt and decrypt operations. The table lists representative requests, the tool and parameters the agent selected, and the outcome that was verified.

Request	Tool selected	Verified outcome
archive ./docs quickly	create_archive (gz, level fast = gzip -1)	docs.tar.gz created next to the source (live demo, Fig. 7.3).
create a zip file of ./src	create_archive (zip)	Valid src.zip; extracts back to identical files (test_zip_round_trip).
compress ./README.md quickly	compress_file (gz, fast)	README.md.gz created; original file untouched.
inspect archive ./docs.tar.gz	inspect_archive	Type and 16 entries reported; nothing written to disk.
what is inside ./demo.zip	list_archive	Entry names listed without extraction.
extract ./backup.tar.xz into ./out	extract_archive	Every member checked, then files restored.
decompress ./data.csv.xz	decompress_file	Original bytes restored exactly (bit-exact test).
Encrypt ./README.md	encrypt_file	README.md.gpg created with GPG symmetric AES-256.

5.1.2 Complex Chained Intents

A chained intent needs several tools in a fixed order, where each step consumes the output of the previous one. The planner builds the whole chain first, shows it with absolute paths, and runs nothing until the user confirms. The first row is the exact prompt from the Agent #12 brief.

Request	Plan produced	Verified outcome
Zip up my project folder, compress it heavily, and encrypt it.	1. create_archive (xz, max = xz -9e) 2. encrypt_file (AES-256) 3. remove_intermediate (plaintext archive)	Only project.tar.xz.gpg remains; decrypting it restores the original files (test_brief_example_end_to_end_and_decrypt; live demo).
decrypt ../project.tar.xz.gpg and extract it	1. decrypt_file 2. extract_archive 3. remove_intermediate	Files restored to project-extracted/; the temporary decrypted archive is deleted.
Archive ./src to ../src.tar.xz and encrypt ../src.tar.xz ../src.tar.xz.gpg	1. create_archive 2. encrypt_file	The user named the intermediate archive, so it is kept.
squeeze ./src as small as possible, then lock it with a password	1. create_archive (max) 2. encrypt_file 3. remove_intermediate	Same chain from different wording.
back up ./project quickly and encrypt it	1. create_archive (gz, fast) 2. encrypt_file	"quickly" selects gzip -1 instead of xz.

	3. remove_intermediate	

If any step fails, the chain stops, already-finished steps are reported (completed_steps), and a failing step leaves no partial output. A wrong passphrase, for example, removes the half-written file and reports "wrong passphrase".

5.1.3 Malicious/Destructive Inputs

Destructive requests are stopped by four independent layers: a regular-expression blacklist applied to the raw text before any planning, the scope check, schema validation of every tool call, and path validation inside each tool (protected system folders, no overwrites, Zip Slip). No request ever reaches a shell, because utilities are started with subprocess.run() and an argument list.

Input	Layer that stopped it	Result
Delete everything on my drive	Guardrail blacklist (mass deletion)	Blocked before planning.
rm -rf /	Guardrail blacklist (rm -rf)	Blocked before planning.
archive ./src; rm -rf ~	Guardrail blacklist (rm -rf, ; injection)	Blocked; the archive part is not run either.
compress $(cat /etc/shadow)	Guardrail blacklist (command substitution)	Blocked.
dd if=/dev/zero of=/dev/sda	Guardrail blacklist (raw disk write)	Blocked.
archive / to ./root.tar.xz	Guardrail blacklist (whole root filesystem)	Blocked.
extract ./sample.tar.xz into /usr/bin	Protected-location check	Refused before anything is written.
Archive containing ../../escaped.txt	Zip Slip check in extract_archive	Refused; destination folder not even created.
Model tool call bash("rm -rf /")	Schema validation in execute_tool	Rejected as an unknown tool; nothing runs.
What is the weather today?	Scope check	Reported as outside this agent's scope.

5.2 Performance Metrics

All metrics were measured with benchmarks/evaluate.py in a single run on the environment above (the run shown in Figure 7.7); the raw numbers are saved in benchmarks/results.json and can be reproduced with python3 benchmarks/evaluate.py.

5.2.1 Intent Translation Accuracy

A request counts as correct only if the agent produces exactly the labelled sequence of tools (and the labelled compression level, where one is given), or blocks / rejects it when that is the expected outcome. Accuracy was measured on separate sets so that tuning does not inflate the result.

Request set	Requests	Correct	Accuracy
Development set (used while building; tuned on)	65	65	100.0%
First held-out set, before the vocabulary rewrite	20	10	50.0%
Second held-out set (written afterwards, never tuned on)	20	16	80.0%

Held-out category	Correct / total	Accuracy
Simple intents	6/10	60.0%
Chained intents	5/5	100.0%
Malicious inputs	3/3	100.0%
Out-of-scope requests	2/2	100.0%

Analysis. The honest estimate of accuracy on new wording is **80%**. The first held-out set scored only 50%: the original keyword list missed everyday verbs ("unlock", "show me the files") and matched "zip" inside "gzip". The vocabulary was rewritten with whole-word matching and a fresh held-out set was written. Its four misses ("turn ./project into a tar.gz", "get the files out of …", "what files does … contain", "check what type of archive … is") were all safe failures: three were rejected as out of scope and one proposed an extra archive that the user would see before confirming. Chained and malicious requests were handled correctly in every set. The weak point is unusual single-verb wording, which is exactly where the Agentic Shell's LLM, calling our tools through their JSON schemas, is expected to help.

5.2.2 Latency (Time-to-Execution)

Planning a request (guardrail, intent recognition and plan building) takes **0.059 ms** on average (95th percentile 0.122 ms), so time-to-execution is dominated by the Linux utilities themselves. Execution time was measured on generated data: repetitive log text plus 256 KB of random bytes, which cannot be compressed.

Input	Format and level	Time	Output size (% of input)
1.25 MB	tar.gz, fast (gzip -1)	9 ms	20.59%
1.25 MB	tar.gz, normal (gzip -6)	11 ms	20.31%
1.25 MB	tar.bz2 (bzip2 -9)	195 ms	20.2%
1.25 MB	tar.xz, normal (xz -6)	70 ms	20.12%

1.25 MB	tar.xz, max (xz -9e)	129 ms	20.12%
1.25 MB	zip (deflate 6)	7 ms	20.3%
10.25 MB	tar.gz, fast (gzip -1)	14 ms	3.06%
10.25 MB	tar.gz, normal (gzip -6)	26 ms	2.77%
10.25 MB	tar.bz2 (bzip2 -9)	1623 ms	2.49%
10.25 MB	tar.xz, normal (xz -6)	188 ms	2.47%
10.25 MB	tar.xz, max (xz -9e)	344 ms	2.47%
10.25 MB	zip (deflate 6)	31 ms	2.77%

End-to-end operation	Time
Brief's example: plan + tar.xz (xz -9e) + GPG AES-256 + clean-up, 1.25 MB folder	0.25 s
GPG AES-256 encryption of a 0.253 MB archive	0.12 s (0.12–0.82 s across runs, depending on whether gpg-agent was already running)

Observations. gzip is the fastest and zip is comparable. bzip2 was the slowest by far (about 1.6 s for 10 MB), which is why the planner never chooses it unless the user asks. xz -9e took about 1.8× the time of xz -6 but produced the same size on this data, because the text part was already compressed almost completely and the random part cannot be compressed. Heavier levels pay off only on data with long-range redundancy.

5.2.3 Error Recovery Rate

In this design the LLM never writes a shell command; it can only propose a call to one of the eight tools. A "syntactically invalid command" therefore appears as an invalid or unsafe tool call. The benchmark sent 18 such calls to execute_tool(). A case counts as recovered only if the agent returned a structured error, raised no exception, created no file and left existing files unchanged.

Invalid or unsafe call	Agent response
Unknown tool name	unknown tool 'delete_files'
Hallucinated shell tool	unknown tool 'bash'
Missing required argument	missing argument 'output_path'
Extra injected argument	unexpected argument 'shell'
Invalid enum value	argument 'level' must be one of ['fast', 'normal', 'max']
Wrong argument type	argument 'archive_path' must be a non-empty string

Empty path	argument 'source_path' must be a non-empty string
Arguments not an object	arguments must be an object
Non-existent source	Path does not exist: <path>
Overwrite existing output	Output already exists: <path>
Write into /etc	Refusing to write into a protected system location: /etc/backup.tar.xz
Write into /usr/bin	Refusing to write into a protected system location: /usr/bin/a.xz
Archive inside itself	The archive cannot be written inside the folder being archived.
Folder passed to compress_file	Compression input must be a file. Folders are archived with create_archive.
Zip Slip archive	Archive contains a path that escapes the destination.
Unsupported decompress format	Cannot infer compression format. Use a .gz, .bz2, or .xz file.
Same input and output	The output path must differ from the source path.
Wrong decryption passphrase	GPG decryption failed. Check the passphrase. (wrong passphrase)

Result: 18/18 = 100.0% recovered. The interactive shell never terminated on an error during testing; every failure is returned as a dictionary with success = false and an error message, and the session continues. The benchmark also caught a real defect: the first version created the extraction folder before scanning a malicious archive, leaving an empty folder behind; the scan now runs before anything is created.

Chapter 6: Conclusion & Future Scope

6.1 Summary of Contributions

The project delivers a working prototype that bridges conversational text and real system execution for the archive domain of the Agentic Shell:

- **Working prototype:** the brief's request "Zip up my project folder, compress it heavily, and encrypt it" runs end to end: tar.xz at xz -9e, GPG AES-256, and removal of the unencrypted copy.
- **LLM-ready tools:** eight tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) with JSON Schema definitions available in generic, OpenAI and Anthropic formats through get_tools(), and one safe entry point, execute_tool(), for the base shell to call.
- **Offline planner:** a deterministic planner that needs no API key or network, chains steps, maps words such as "heavily" and "quickly" to compression levels, chooses safe default file names, and cleans up intermediate plaintext files.

- **Layered safety:** destructive-command blacklist, scope check, schema validation, human confirmation, protected system folders, Zip Slip prevention, no overwrites, and passphrases passed to gpg on standard input.
- **Ready for integration:** installable with one pip command from GitHub, standard library only, a unique package name (smart_archive_agent) and prefixed tool names so it can be loaded next to the other 34 agents, a machine-readable manifest (agent.json) for the shell's router, an integration guide with a runnable example, and a --doctor check for the required Linux tools.
- **Measured quality:** 58 automated tests, 80% intent accuracy on unseen phrasings, sub-millisecond planning, and 100.0% recovery from invalid tool calls.

6.2 Challenges Encountered

- **Managing state and context history across multiple command executions:** within one request the agent passes each step's output path to the next step and records which files it created, so the clean-up step can delete only those files and a failed step can report exactly how far the chain got. Across separate requests the agent keeps no conversation history, so a follow-up such as "now encrypt it" must name the file. This keeps behaviour predictable and safe, but it limits natural multi-turn conversation.
- **Generalising beyond fixed keywords:** the first held-out test scored 50%, and "zip" was matched inside "gzip". Rewriting the vocabulary with whole-word matching raised unseen-phrasing accuracy to 80%, but rule-based recognition still cannot cover every wording.
- **GPG inside a subprocess:** the first version failed with "Inappropriate ioctl for device" because gpg tried to open its own passphrase prompt. It now uses loopback pinentry, reads the passphrase from standard input, and disables passphrase caching so a wrong passphrase is always detected.
- **Ordering of safety checks:** Zip Slip detection originally ran after the destination folder was created; the error-recovery benchmark exposed this and the check now runs first.
- **Running on other machines:** the agent must work on the teacher's machine, not only ours. The package was renamed from the generic name app, the code uses only the standard library, and --doctor lists missing tools with the exact apt command. Only Python 3.14 was available for testing; older versions are not yet verified.

6.3 Future Work

- **Multi-modal support:** voice-controlled requests through local speech-to-text (for example Whisper or Vosk) feeding the same planner and confirmation step.
- **Deeper local file indexing:** an index of file types and modification times so requests such as "archive all spreadsheets from last quarter" can select their sources, possibly reusing the Semantic File Search agent.
- **Conversation context:** remembering the last output so that follow-ups like "now encrypt it" work, while still showing every plan before it runs.
- **MCP server:** exposing the same tool schemas through a Model Context Protocol server so any MCP client can use the agent.

- **Adaptive compression:** measuring file entropy first and skipping compression for data that is already compressed (JPEG, MP4, PDF), as the latency results suggest.
- **Wider platform testing:** running the test suite on Python 3.8–3.13 and on older Ubuntu releases used in the lab.

Chapter 7: Screenshots

The figures below are the terminal output of the demonstration, captured on Ubuntu 26.04.1 LTS (WSL 2) on 22 October 2026 and shown as terminal images. Each figure begins with the command or request that was typed; passphrases do not appear because they are read with hidden input. A small demo folder is used so the archive is quick to create. It was prepared with:

```
$ mkdir -p ~/demo/project && cp -r ~/SMART-ARCHIVE-AGENT/docs ~/demo/project/
$ cd ~/demo/project && export PYTHONPATH=~/SMART-ARCHIVE-AGENT
```

7.1 Environment Check

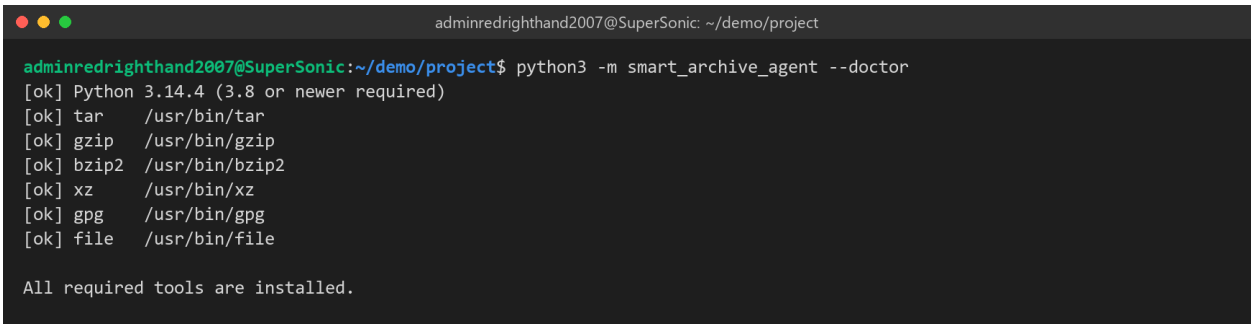


Figure 7.1: Dependency check: Python and tar, gzip, bzip2, xz, gpg and file found.

7.2 Brief's Example: Plan, Confirmation and Encryption

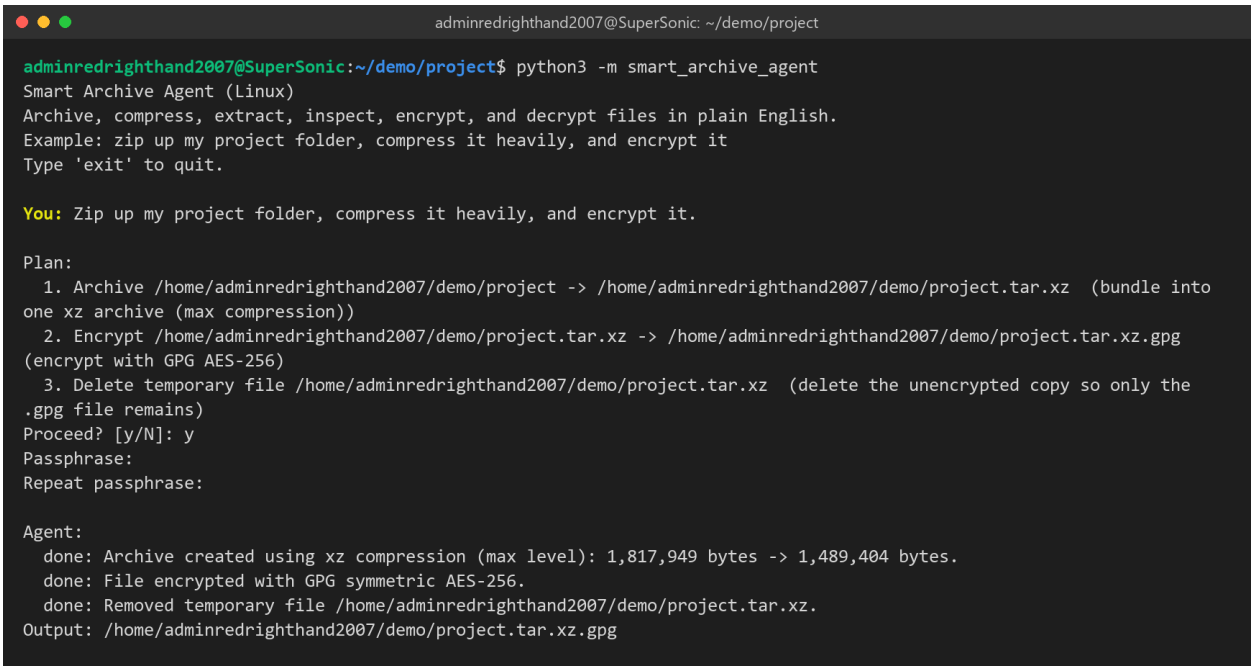


Figure 7.2: Three-step plan shown before execution; archive at xz -9e, GPG AES-256, plaintext copy removed.

7.3 Fast Compression and Inspection

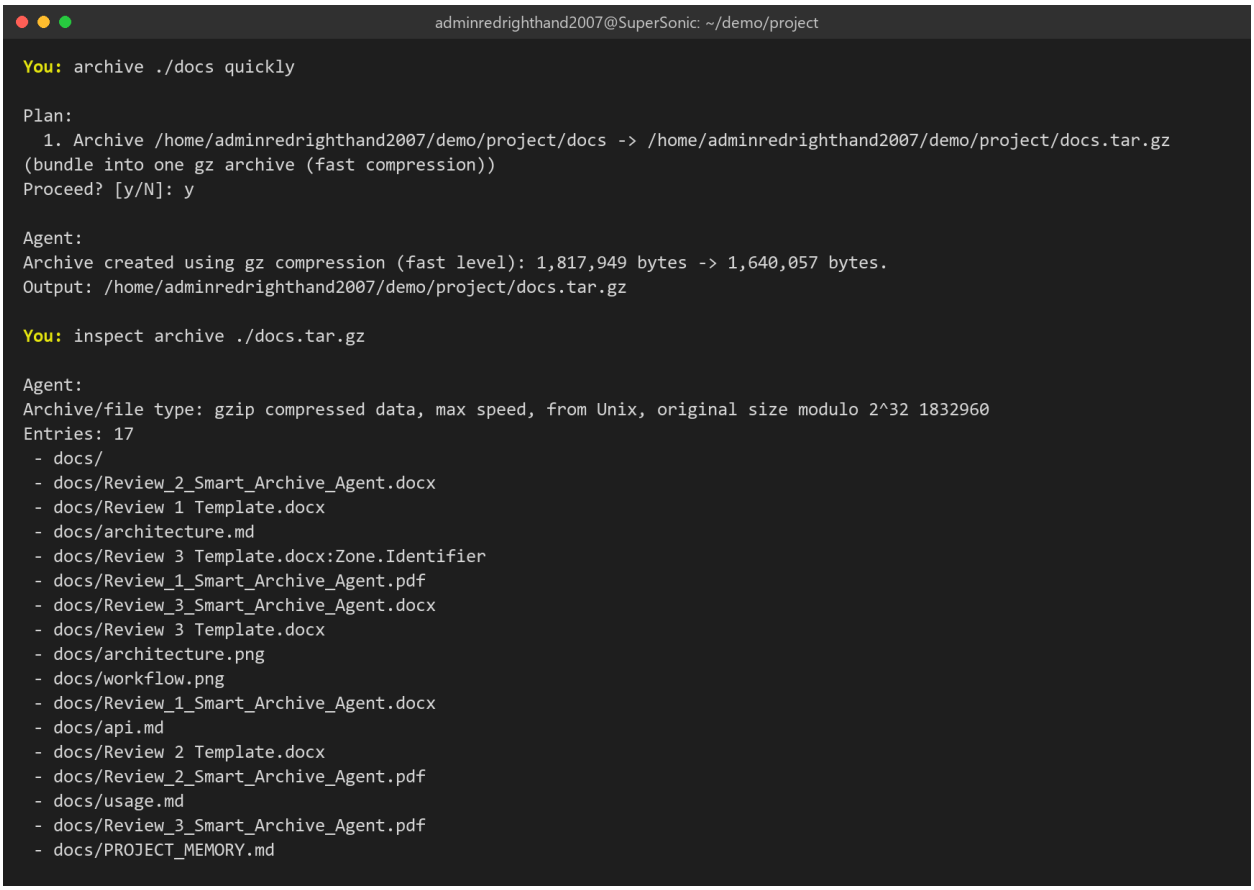


Figure 7.3: "quickly" selects gzip -1; inspection lists the archive without extracting it.

7.4 Decrypting and Extracting in One Request



Figure 7.4: Decrypt, extract and clean-up chained from one sentence.

7.5 Guardrails

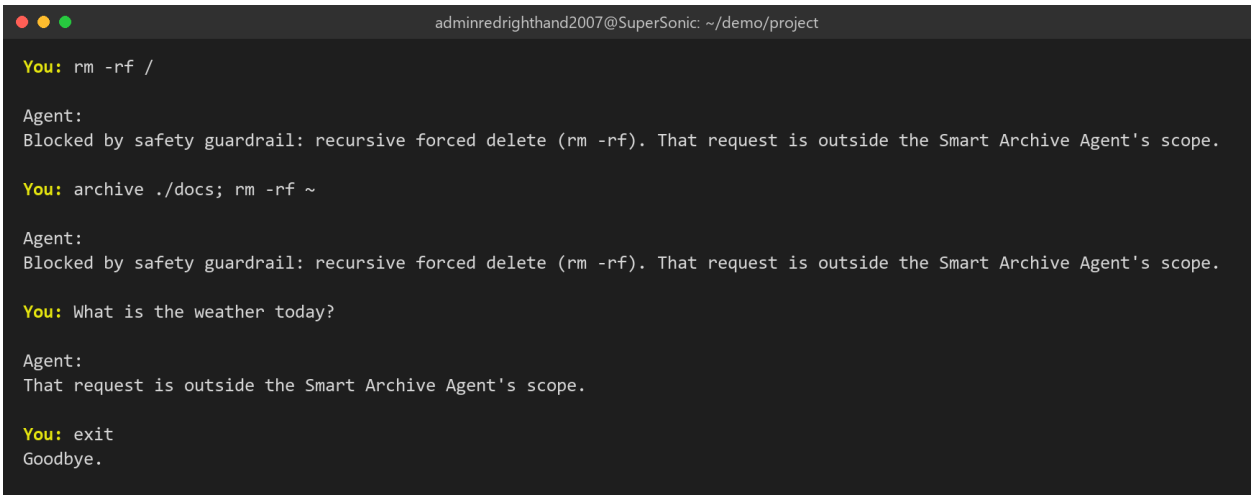


Figure 7.5: Destructive requests blocked before planning; an unrelated request reported as out of scope.

7.6 Automated Test Suite

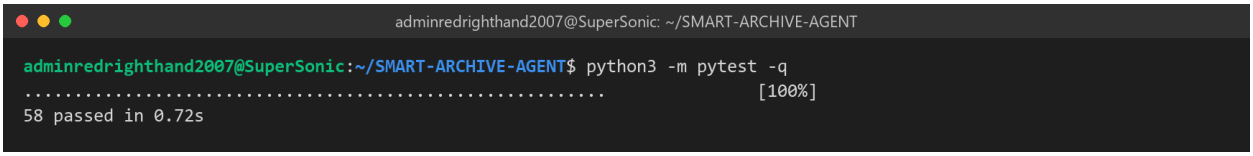


Figure 7.6: 58 tests passed.

7.7 Benchmark Results

```
adminredrighthand2007@SuperSonic: ~/SMART-ARCHIVE-AGENT

adminredrighthand2007@SuperSonic:~/SMART-ARCHIVE-AGENT$ python3 benchmarks/evaluate.py
Intent accuracy (development set): 65/65 = 100.0%
  simple        30/30 = 100.0%
  chained       15/15 = 100.0%
  malicious     13/13 = 100.0%
  out_of_scope  7/7 = 100.0%
  planning latency: mean 0.059 ms, p95 0.122 ms
Intent accuracy (held-out set): 16/20 = 80.0%
  simple        6/10 = 60.0%
  chained       5/5 = 100.0%
  malicious     3/3 = 100.0%
  out_of_scope  2/2 = 100.0%
  MISS: {'request': 'turn ./project into a tar.gz', 'expected': ['create_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'get the files out of ./sample.tar.xz', 'expected': ['extract_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'what files does ./sample.zip contain', 'expected': ['list_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'check what type of archive ./sample.tar.xz is', 'expected': ['inspect_archive'], 'got':
['create_archive'], 'expected_level': None, 'got_level': 'normal'}
  planning latency: mean 0.056 ms, p95 0.125 ms
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'fast', 'ms': 8.7, 'ratio_pct': 20.59}
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'normal', 'ms': 10.9, 'ratio_pct': 20.31}
  exec {'input_mb': 1.25, 'format': 'bz2', 'level': 'normal', 'ms': 195.1, 'ratio_pct': 20.2}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'normal', 'ms': 70.4, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'max', 'ms': 128.7, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'zip', 'level': 'normal', 'ms': 7.3, 'ratio_pct': 20.3}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'fast', 'ms': 13.5, 'ratio_pct': 3.06}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'normal', 'ms': 26.5, 'ratio_pct': 2.77}
  exec {'input_mb': 10.25, 'format': 'bz2', 'level': 'normal', 'ms': 1623.2, 'ratio_pct': 2.49}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'normal', 'ms': 188.1, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'max', 'ms': 343.5, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'zip', 'level': 'normal', 'ms': 31.3, 'ratio_pct': 2.77}
  exec {'input_mb': 0.253, 'format': 'gpg-aes256', 'level': '-', 'ms': 120.3, 'ratio_pct': None, 'success': True}
  exec {'input_mb': 1.25, 'format': 'chain: tar.xz max + gpg + cleanup', 'level': 'max', 'ms': 245.7, 'ratio_pct': None,
 'success': True}
Error recovery: 18/18 = 100.0%
Saved /home/adminredrighthand2007/SMART-ARCHIVE-AGENT/benchmarks/results.json
```

Figure 7.7: Intent accuracy, latency and error-recovery measurements used in Section 5.2.

References

[1] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[2] Tukaani Project, "XZ Utils," documentation. Available: https://tukaani.org/xz/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[5] Python Software Foundation, "tarfile — Extraction filters" and PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

[6] Snyk Security Research, "Zip Slip Vulnerability," 2018. Available: https://security.snyk.io/research/zip-slip-vulnerability

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/